

ДИСЦИПЛИНА	Технологии разработки серверных приложений
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям
ПРЕПОДАВАТЕЛЬ	Дворецкий Артур Геннадьевич
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Ссылка на материал:

https://github.com/dvOretsky/fastapi-tutorial/blob/main/FAPI1_Introduction/FAPI1_Introduction.md

Практическое занятие №1: Введение в FastAPI

FastAPI - это современный веб-фреймворк, который помогает разработчикам быстро и эффективно создавать API*. Он основан на Python 3.5+ type hints**, что позволяет ему обеспечивать автоматическую проверку данных, сериализацию и документирование с использованием моделей **Pydantic**. Одним из наиболее привлекательных аспектов **FastAPI** является его впечатляющая производительность: он способен обрабатывать тысячи запросов в секунду с минимальными накладными расходами, что делает его отличным выбором для приложений с высоким трафиком.

*Прочитать что такое API можно тут: <https://habr.com/ru/articles/464261/>

**стоит отметить, что в Python type hints в современном виде появились с релизом 3.5 (PEP 484), однако на официальной страничке FastAPI на [pyd.org](https://fastapi.tiangolo.com/) указано, что оно базируется на Python 3.7+.

Подробнее про подсказки типов можно прочитать на хабре:
<https://habr.com/ru/companies/lamoda/articles/432656/>

Ключевые особенности FastAPI:

Поддержка асинхронности: **FastAPI** полностью поддерживает асинхронное программирование, позволяя вам писать асинхронные обработчики маршрутов и использовать преимущества синтаксиса `async` / `await` в Python для неблокирующих операций ввода-вывода.

Автоматическое документирование: **FastAPI** автоматически генерирует интерактивную и удобную для пользователя документацию по **API**. Он использует стандарты **OpenAPI** и **JSON Schema** для предоставления исчерпывающей документации для вашего **API**, включая проверку входных данных, ожидаемые ответы и многое другое.

Подсказки типов и проверка данных: Благодаря интеграции **FastAPI** с **Pydantic** вы можете определять модели данных с помощью подсказок типов (type hints) Python, обеспечивая автоматическую проверку данных и сериализацию. Эта функция помогает выявлять ошибки на ранних стадиях процесса разработки и повышает читаемость кода.

Внедрение зависимостей: **FastAPI** поддерживает внедрение зависимостей, позволяя вам эффективно управлять зависимостями и организовывать их. Эта функция особенно полезна при работе с подключениями к базе данных, аутентификацией и другими общими ресурсами.

Простой и интуитивно понятный синтаксис: синтаксис **FastAPI** понятен, легок для чтения и очень похож на стандартное определение функций Python, что делает его доступным как для начинающих, так и для опытных разработчиков.

Почему именно FastAPI?

Давайте посмотрим, какие альтернативы были (и есть) у **FastAPI**, чтобы понять почему именно **FastAPI** и какую боль выбранный вами для изучения фреймворк закрывает:

1. **Flask** - это легкий и широко используемый веб-фреймворк в экосистеме Python. Он прост в использовании и с ним легко начать работу, но ему не хватает встроенной поддержки асинхронности и автоматической проверки на основе подсказок типов.
2. **Django** - это полнофункциональный веб-фреймворк, который следует философии "батарейки включены" (швейцарский нож - все в комплекте). Он предоставляет множество готовых функциональных возможностей, включая ORM, интерфейс администратора и многое другое, но для небольших и простых **API** это может оказаться излишним. В дополнение можно поругать джангу, тем что в нем зашит паттерн MVC (model-view-controller, или model-view-template по-джанговски), а также это по сути модульный монолит и он не такой гибкий, как **FastAPI**.

В целом это топ-2, которые носили реально массовый характер. Описанные ниже фреймворки были и есть не очень популярны, не очень удобны, либо не очень производительны.

3. **Tornado** - это асинхронный веб-фреймворк, который может обрабатывать большое количество одновременных подключений. Он часто используется в сценариях, требующих высокого уровня параллелизма, но может иметь более крутую кривую обучения по сравнению с **FastAPI** (тяжел, местами не очевиден, не очень удобен).
4. **Bottle** - это минималистичный веб-фреймворк, разработанный для маломасштабных приложений. Он легкий и простой в использовании, но ему не хватает скорости, масштабируемости и оптимизации производительности, присутствующих в **FastAPI**.

Основные плюсы **FastAPI**:

- **скорость** (питон всегда считался медленным, но API на FastAPI сопоставим с языком Go (язык гугла, в котором изначально зашита асинхронность и скорость работы) или Node.js (довольно быстрым и популярным сейчас в российском бэкенде). По итогу **fastapi** является на данный момент самым быстрым веб-фреймворком питона (он где-то в 4-5 раз быстрее джанги и фласка - его основных конкурентов).
- **встроенная поддержка проверки типов** (питон это динамический язык со строгой типизацией, что означает возможность преобразовать один тип данных в другой, но строгость означает, что питон не делает это преобразование неявно; проблема у разработчика-питониста была в том, что иногда можно было поймать неожиданный тип данных, из-за чего

программа могла лечь). **FastAPI** эту проблему снимает (при помощи Pydantic'a, о котором расскажем дальше в курсе).

- **асинхронность** "из коробки", то есть на нем можно писать реально высоконагруженные API-шки.

Установка FastAPI:

Чтобы начать работу с **FastAPI**, вам нужно будет установить его с помощью **pip**. Откройте свой терминал или командную строку и введите следующую команду:

```
pip install fastapi[all]
```

Для запуска FastAPI нужен сервер, поэтому вам также нужно установить "**uvicorn**" (это молниеносный сервер **ASGI**, для эффективного запуска ваших приложений **FastAPI**), точнее проверить, что все установилось корректно:

```
pip install uvicorn[standard]
```

Для быстрого старта этого будет достаточно, а про альтернативы единорожке **uvicorn'y** скажем пару слов в конце семестра (спойлер: разница будет становиться ощутима уже на продвинутом уровне, когда вам нужно будет решать, как приделать конкретный фреймворк к вашему приложению).

Подробнее про **ASGI** можно прочитать тут: <https://habr.com/ru/articles/482936/>

Также можно погуглить отличие классических **WSGI** от **ASGI**, чтобы увидеть преимущества последнего (если кратко - первый это синхронный, а второй асинхронный).

"Привет, МИРЭА!" с помощью FastAPI:

Давайте начнем с создания простого приложения "Привет, МИРЭА!" с помощью **FastAPI**. Укутайтесь теплым пледом, налейте себе вкусного чая, откройте свой любимый текстовый редактор или **IDE** и создайте новый файл с именем **main.py**. Добавьте следующий код:

```
# main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def root():
    return {"message": "Hello MIREA!"}
```

Сохраните файл и запустите приложение с помощью **uvicorn** (в консоли):

```
uvicorn main:app --reload
```

Откройте 'http://localhost:8000' в вашем веб-браузере, и вы должны увидеть сообщение "Hello MIREA!" в формате JSON.

Если создать такое приложение:

```
# обновляем код в main.py
from fastapi import FastAPI

my_awesome_api = FastAPI()

@my_awesome_api.get("/")
async def root():
    return {"message": "Hello MIREA!"}
```

То его вызов будет таким:

```
'uvicorn main:my_awesome_api --reload'
```

Примечание: структура проекта, его настройка и другие важные темы будут раскрыты далее по курсу, сейчас мы просто узнали, что увидеть "Hello MIREA!" очень просто!

Теперь у вас есть базовое представление о том, что такое FastAPI и что он может предложить!

Настройка проекта FastAPI

В этом занятии мы продолжим знакомство с **FastAPI**, но теперь сосредоточимся на организации проекта и создании базовой структуры. Здесь мы научимся правильно настраивать проект, организовывать файлы и каталоги, а также запускать сервер.

В ходе занятия мы:

- Устроим проект с правильно организованной структурой.
- Добавим несколько маршрутов для обработки различных запросов.
- Научимся использовать **Pydantic-модели** для валидации данных.
- Рассмотрим, как настраивать проект для продакшн-среды и работы с переменными окружения.

Структура проекта

Хорошо организованная структура проекта – это залог того, что ваш код будет легко поддерживать и масштабировать. Давайте разберемся, как лучше организовать файлы и каталоги в проекте **FastAPI**.

Вот пример базовой структуры проекта:

```
project_name/
├── app/
│   ├── __init__.py
│   ├── main.py
│   └── models/
│       └── __init__.py
└── requirements.txt
```

- Каталог **app/** содержит основной модуль приложения и вложенные пакеты для моделей и маршрутов.
- Файл **main.py** – точка входа в ваше приложение **FastAPI**. Здесь будут определяться маршруты и запускаться сервер.
- Каталог **models/** используется для размещения моделей данных, например, моделей Pydantic для обработки запросов и ответов.

Для начала нам нужно создать файлы **main.py** и **requirements.txt**. В **requirements.txt** будут указаны все зависимости, такие как **FastAPI** и **Uvicorn**, а в **main.py** будет размещаться основное приложение и маршруты.

Зачем такая структура? Если проект вырастет, это позволит вам легко добавлять новые функциональности и делить код на логичные части, например, по модулям или сервисам.

Разнообразие структур проектов FastAPI

При разработке приложений на **FastAPI** существуют различные подходы к организации структуры проекта. Рассмотрим основные из них:

1. Функциональная структура

В этой модели проект организуется вокруг основных функций модулей. Например:

```
project/
├── app/
│   ├── api/
│   │   ├── __init__.py
│   │   ├── endpoints/
│   │   │   ├── __init__.py
│   │   │   ├── tasks.py
│   │   │   └── users.py
│   │   └── schemas/
│   │       ├── __init__.py
│   │       ├── task.py
│   │       └── user.py
│   └── core/
│       ├── __init__.py
│       ├── config.py
│       └── security.py
```

```

├── db/
│   ├── __init__.py
│   ├── models.py
│   └── database.py
├── middleware/
│   ├── __init__.py
│   └── middlewarees.py
├── errors/
│   ├── __init__.py
│   ├── exceptions.py
│   └── handlers.py
├── metrics/
│   ├── __init__.py
│   └── prometheus.py
├── __init__.py
├── tests/
│   ├── __init__.py
│   ├── test_endpoints.py
│   └── test_models.py
├── .env
├── .gitignore
├── main.py
├── pyproject.toml
├── README.md
└── requirements.txt

```

Преимущества:

- Логическая группировка: Файлы организованы по функциональным областям, что упрощает понимание и поддержку кода.
- Масштабируемость: Легко добавлять новые функции, расширяя соответствующие каталоги.

Недостатки:

- Потенциальная избыточность: При большом количестве функций структура может стать сложной и трудоемкой в управлении.
- Сложность навигации: Требуется привыкание к разделению по функциональным областям.

2. Сервисно-ориентированная структура

В этой модели проект делится на отдельные сервисы, каждый из которых отвечает за конкретную часть функциональности. Например:

```

project/
├── app/
│   ├── users_service/
│   │   ├── __init__.py
│   │   ├── main.py
│   │   ├── models.py
│   │   └── routers/
│   │       └── users.py

```

```

├── tasks_service/
│   ├── __init__.py
│   ├── main.py
│   ├── models.py
│   └── routers/
│       └── tasks.py
├── __init__.py
├── tests/
│   ├── __init__.py
│   ├── test_users.py
│   └── test_tasks.py
├── .env
├── .gitignore
├── main.py
├── pyproject.toml
├── README.md
└── requirements.txt

```

Преимущества:

- Изоляция сервисов: Каждый сервис независим, что облегчает тестирование и поддержку.
- Гибкость: Легко заменять или обновлять отдельные сервисы без влияния на остальные.

Недостатки:

- Сложность интеграции: Необходима продуманная система взаимодействия между сервисами.
- Увеличение нагрузки: Множество сервисов может привести к дополнительным накладным расходам на управление.

3. Слойная структура

В этой модели проект разделяется на слои, отражающие различные уровни приложения, такие как:

- **API слой:** Обработка запросов и маршрутизация.
- **Сервисный слой:** Бизнес-логика.
- **Слой доступа к данным:** Взаимодействие с базой данных.

Пример структуры:

```

project/
├── app/
│   ├── api/
│   │   ├── __init__.py
│   │   └── routers/
│   │       └── users.py
│   ├── services/
│   │   ├── __init__.py
│   │   └── user_service.py
│   └── data/
│       ├── __init__.py
│       └── repositories/

```

```
├── __init__.py
├── user_repository.py
├── models/
│   ├── __init__.py
│   └── user.py
├── __init__.py
├── tests/
│   ├── __init__.py
│   ├── test_users.py
│   └── test_services.py
├── .env
├── .gitignore
├── main.py
├── pyproject.toml
├── README.md
└── requirements.txt
```

Преимущества:

- Четкое разделение ответственности: Каждый слой имеет свою четко определенную роль.
- Упрощение тестирования: Легко тестировать отдельные слои независимо.

Недостатки:

- Возможная избыточность: Для небольших проектов разделение на слои может быть чрезмерным.
- Необходимость тщательного планирования: Требуется продуманная архитектура для эффективного взаимодействия между слоями.

Выбор структуры проекта зависит от множества факторов, включая размер проекта, команду разработки и специфические требования. Рекомендуется:

- **Оценить масштаб проекта:** Для небольших приложений может быть достаточно простой структуры, тогда как крупные проекты потребуют более сложной архитектуры.
- **Учитывать командную работу:** Если над проектом работает несколько человек, важно выбрать структуру, которая облегчает совместную работу и понимание кода.
- **Обращать внимание на масштабируемость:** Структура должна позволять легко добавлять новые функции и поддерживать рост проекта.

Подробное руководство по организации структуры проекта можно найти, например, в статьях на Habr'e:

<https://habr.com/ru/articles/871018/>

<https://habr.com/ru/articles/827222/>

Настройка виртуального окружения

Для того чтобы обеспечить изоляцию зависимостей проекта и избежать конфликтов между библиотеками, все установленные пакеты должны находиться в виртуальном окружении.

Виртуальное окружение – это независимая среда, в которой ваш проект использует только те библиотеки, которые ему нужны, без влияния на глобально установленные пакеты в системе.

Почему это важно?

- **Изоляция зависимостей:** Каждый проект может использовать разные версии библиотек. Виртуальное окружение позволяет установить именно те версии, которые необходимы вашему проекту.
- **Удобство поддержки:** Легко переносить проект на другой компьютер или сервер, установив только нужные зависимости.
- **Безопасность:** Глобальные изменения в системе не затрагивают ваше приложение, а обновления или эксперименты с пакетами не приводят к неожиданным конфликтам.

Как создать виртуальное окружение

На Windows

1. Откройте командную строку или **PowerShell**.
2. Перейдите в корневую папку вашего проекта.
3. Выполните команду для создания виртуального окружения:

```
python -m venv venv
```

Здесь **venv** – имя папки, в которой будет создано виртуальное окружение.

4. Активируйте виртуальное окружение:

- В командной строке:

```
venv\Scripts\activate
```

- В PowerShell:

```
.\venv\Scripts\Activate.ps1
```

На macOS и Linux

1. Откройте терминал.
2. Перейдите в корневую папку вашего проекта.
3. Создайте виртуальное окружение:

```
python3 -m venv venv
```

4. Активируйте виртуальное окружение:

```
source venv/bin/activate
```

После активации виртуального окружения в командной строке или терминале обычно появляется префикс (например, `(venv)`), указывающий на активированную среду.

Установка зависимостей

После активации виртуального окружения установите все необходимые пакеты, указанные в файле `requirements.txt`. Например:

```
pip install -r requirements.txt
```

Это гарантирует, что все зависимости будут установлены в изолированной среде, и ваш проект не будет конфликтовать с другими проектами или системными библиотеками.

Запуск приложения и точки входа

Теперь давайте сосредоточимся на запуске вашего FastAPI-приложения и разберемся, что такое точка входа в приложение, как правильно запускать проект и какие инструменты можно использовать для тестирования.

Создание приложения FastAPI

Если вы пропустили первый урок, создайте базовое приложение **FastAPI** в файле `main.py`. Для этого откройте файл `main.py` в вашем проекте и добавьте следующий код:

```
# main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello, MIREA!"}
```

Здесь мы создаем объект приложения `app` с помощью **FastAPI** и добавляем простой маршрут с методом **GET**, который будет возвращать сообщение `"Hello, MIREA!"` при переходе по корневому URL `/`.

Запуск приложения

После того как вы написали код, пришло время запустить приложение. **FastAPI** использует сервер **uvicorn** для запуска приложения. Чтобы запустить его, откройте терминал и выполните команду:

```
uvicorn main:app --reload
```

Эта команда делает следующее:

- **main**: указывает на файл **main.py**.
- **app**: указывает на объект **FastAPI**, который мы создали.
- **--reload**: позволяет автоматически перезапускать сервер при изменениях в коде, что удобно во время разработки.

Если начали внедрять структуру проекта, то есть создали папку **app** внутри директории проекта **my_project**, то команда запуска приложения будет такой:

```
uvicorn app.main:app --reload
```

Где первый **app** - это папка с файлом **main.py**.

FastAPI также предоставляет собственный инструмент командной строки для запуска приложений. Для этого выполните:

```
fastapi dev main.py
```

Эта команда автоматически найдет объект **app** в вашем файле **main.py** и запустит сервер.

Примечание: Если вы используете **FastAPI CLI** в production-режиме, рекомендуется заменить **dev** на **run**.

После запуска приложения вы должны увидеть вывод в консоли, который подтверждает, что сервер запущен и работает на адресе: **http://localhost:8000**.

Точка входа

Точка входа в приложение — это место, где начинается выполнение кода. В случае с **FastAPI** точка входа — это файл, в котором вы создаете объект приложения **FastAPI()**, и с которого запускается сервер. Обычно это файл **main.py**, и именно в этом файле мы будем размещать маршруты и всю основную логику приложения.

Доступ к API и документация

Когда сервер запущен, вы можете открыть веб-браузер и перейти по адресу:

- **http://localhost:8000** — это будет ваша точка входа, которая возвращает сообщение **{"message": "Hello, MIREA!"}**.

Кроме того, **FastAPI** автоматически генерирует интерактивную документацию, которая поможет вам тестировать **API** прямо из браузера. Для этого просто откройте:

- <http://localhost:8000/docs> — здесь вы увидите интерфейс **Swagger UI**, с помощью которого можно отправлять запросы к вашему **API**.
- <http://localhost:8000/redoc> — это альтернативный интерфейс для документации.

Если вы хотите отправлять запросы не через браузер, а через инструменты, такие как **Postman**, вы также можете использовать их для тестирования **API**. **Postman** позволяет отправлять запросы (например, **GET** или **POST**) на ваш сервер и просматривать ответы.

Теперь вы можете переходить к созданию более сложных маршрутов и обработке данных, но сначала убедитесь, что приложение запускается корректно и доступно через браузер.

Добавление дополнительных маршрутов

FastAPI позволяет вам определить несколько маршрутов для обработки различных конечных точек в вашем приложении. Давайте добавим новый маршрут, который возвращает кастомное сообщение.

1. Откройте файл `main.py` и обновите его следующим кодом:

```
# обновляем код main.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello, World!"}

# новый маршрут
@app.get("/custom")
def read_custom_message():
    return {"message": "This is a custom message!"}
```

В этом примере мы добавили новый маршрут `/custom`, который обрабатывает GET-запросы и возвращает сообщение `"This is a custom message!"`.

2. После сохранения изменений перезагрузите сервер (если стартовали ранее без флага `reload`):
 - Нажмите `Ctrl + C`, чтобы остановить предыдущий экземпляр.
 - Затем снова запустите сервер командой:

```
uvicorn main:app --reload
```

3. Теперь, если вы перейдете по адресу <http://127.0.0.1:8000/custom>, вы получите ответ:

```
{  
  "message": "This is a custom message!"  
}
```

Pydantic-модели для аннотации типов в FastAPI

Pydantic — это Python-библиотека для выполнения валидации данных.

Вы объявляете «форму» данных как классы с атрибутами, и каждый атрибут имеет тип. Затем вы создаёте экземпляр этого класса с некоторыми значениями, и **Pydantic** проверяет эти значения, преобразует их в соответствующий тип (если данные корректны) и предоставляет вам объект со всеми данными.

Пример создания модели

```
# вставьте этот код в любой файл, который бы вы могли выполнить  
# например это abc.py, который вы запустите в терминале командой python3  
abc.py  
from datetime import datetime  
from pydantic import BaseModel  
  
# Создаём модель данных, которая обычно располагается в файле models.py  
class User(BaseModel):  
    id: int  
    name: str = "John Doe"  
    signup_ts: datetime | None = None  
    friends: list[int] = []  
  
# Внешние данные, имитирующие входящий JSON  
external_data = {  
    "id": "123",  
    "signup_ts": "2017-06-01 12:22",  
    "friends": [1, "2", b"3"],  
}  
  
# Имитация распаковки входящих данных в коде приложения  
user = User(**external_data)  
print(user)  
# > User id=123 name='John Doe' signup_ts=datetime.datetime(2017, 6, 1,  
12, 22) friends=[1, 2, 3]  
print(user.id)  
# > 123
```

В этом примере **Pydantic** автоматически преобразует строку "123" в целое число, дату в объект `datetime`, а элементы списка, такие как "2" или b"3", в целые числа.

Использование Pydantic в FastAPI

С помощью аннотаций типов в **FastAPI** вы объявляете параметры запроса, тела запроса, заголовки, зависимости и другие элементы. При этом **Pydantic** используется для:

- **Поддержки редактора.** Благодаря аннотациям типов **IDE** могут предоставлять автодополнение и проверку кода.
- **Проверки типов.** Входящие данные автоматически преобразуются к нужным типам.
- **Валидации данных.** Если данные не соответствуют объявленным типам, **FastAPI** автоматически генерирует ошибку и возвращает её клиенту.
- **Документирования API.** Используя OpenAPI, **FastAPI** создает интерактивную документацию, где указаны все требования к входящим данным.

Еще один пример использования модели

```
from datetime import date
from pydantic import BaseModel

# Функция с аннотацией типа параметра, что позволяет получать поддержку
# проверки типов в IDE
def main(user_id: str):
    return user_id

# Еще один пример создания модели Pydantic
class User(BaseModel):
    id: int
    name: str
    joined: date

# Модель User может быть использована для создания объектов и распаковки
# JSON-запросов:
my_user: User = User(id=3, name="John Doe", joined="2018-07-19")

second_user_data = {
    "id": 4,
    "name": "Mary",
    "joined": "2018-11-30",
}

my_second_user: User = User(**second_user_data)
```

Пример реального использования модели с FastAPI

```
# main.py
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class User(BaseModel):
    username: str
```

```
message: str

@app.post("/")
async def root(user: User):
    """
    Здесь мы можем с переменной user, которая содержит объект класса User
    с соответствующими полями,
    выполнить любую логику – например, сохранить информацию в базу данных,
    передать в другую функцию и т.д.
    """
    print(f'Мы получили от юзера {user.username} такое сообщение:
    {user.message}')
    return user
```

При отправке POST-запроса с JSON-данными по корневому адресу **FastAPI** автоматически проверит соответствие входных данных объявленной модели и вернет ошибку, если что-то не соответствует.

Пример корректного POST-запроса (JSON)

```
{
  "username": "Artur",
  "message": "I am Spider-man!"
}
```

Примеры некорректных запросов, вызывающих ошибки

При получении невалидных данных **Pydantic** генерирует ошибку, и **FastAPI** возвращает клиенту подробное сообщение об ошибке.

Пример 1:

```
{
  1: 12345
}
```

Комментарий: Здесь ключ не является строкой (в JSON ключи должны быть строками), поэтому данные не будут корректно обработаны.

Пример 2:

```
{
  "username": "stas_norman",
  "message": false
}
```

Комментарий: Значение `false` не является строкой, а модель ожидает тип `str` для поля `message`.

Пример 3:

```
{  
  "username": "rustam"  
}
```

Комментарий: Поле `message` отсутствует, а оно является обязательным, что вызовет ошибку валидации.

Зачем нужна валидация входных данных?

Проверка входных данных необходима для обеспечения того, чтобы получаемые данные соответствовали вашим ожиданиям. Например:

- Если вы ожидаете строку для хранения в базе данных (например, поле `VARCHAR (255) NOT NULL`), а получаете число или другой тип – это может привести к ошибкам.
- Если ваше **API** предназначено для выполнения арифметических операций (например, возведения числа в квадрат), а приходит строка или `None`, попытка выполнить операцию вызовет сбой.
- Если вы ожидаете массив с определенным количеством элементов, а получаете меньше – попытка доступа к несуществующему элементу приведет к ошибке.

Таким образом, валидация данных помогает предотвратить сбои в работе приложения, обеспечивая корректную обработку входящих данных.

Прочие настройки проекта

В этом шаге мы рассмотрим, как настроить ваш проект для работы в разных средах. Это особенно важно для продакшн-режима, когда нужно учитывать переменные окружения, логирование и конфигурационные файлы. Мы покажем, как хранить конфиденциальные данные (например, ключи **API** и данные подключения к базе данных) вне исходного кода и как структурировать настройки проекта для удобства поддержки.

Переменные окружения

Переменные окружения позволяют хранить чувствительные данные отдельно от кода. Это удобно и безопасно: вы можете менять настройки для разработки, тестирования и продакшн-окружения без изменения исходного кода.

1. Установка библиотеки

Мы будем использовать библиотеку `enviorns`, которая помогает загружать переменные окружения из файла `.env` и выполнять валидацию типов данных. Это современная альтернатива `python-dotenv`.


```
pip install environs
```

2. Создание файла `.env`

Создайте файл `.env` в корне проекта и добавьте в него, например, следующие настройки:

```
# .env
DATABASE_URL=postgresql://user:password@localhost/mydatabase # просто для
примера, ставить сейчас постгрес не нужно
SECRET_KEY=mysecretkey
DEBUG=True
```

Важно: Файл `.env` не должен попадать в систему контроля версий (например, в Git).
Добавьте его в `.gitignore`.

Также тут и далее по курсу, все комментарии из `ini` и `env` файлов следует удалять, чтобы нормально работал их парсинг (оставлять можно только в питонячьих файлах).

Использование датаклассов для конфигурации

Чтобы структурировать и централизовать настройки проекта, мы будем использовать **dataclasses**.

Dataclass — это удобный способ в **Python** определить класс, основное предназначение которого — хранение данных. С помощью декоратора `@dataclass` **Python** автоматически сгенерирует такие методы, как `__init__` и `__repr__`, что упрощает создание и использование классов для хранения конфигурационных данных.

1. Создание файла `config.py`

В этом файле мы создадим классы для хранения настроек, а затем функцию для их загрузки из переменных окружения:

```
# config.py
from dataclasses import dataclass
from environs import Env

@dataclass
class DatabaseConfig:
    database_url: str

@dataclass
class Config:
    db: DatabaseConfig
    secret_key: str
    debug: bool

def load_config(path: str = None) -> Config:
    env = Env()
```

```
env.read_env(path) # Загружаем переменные окружения из файла .env

return Config(
    db=DatabaseConfig(database_url=env("DATABASE_URL")),
    secret_key=env("SECRET_KEY"),
    debug=env.bool("DEBUG", default=False)
)
```

В этом коде:

- Мы определяем класс `DatabaseConfig` для хранения URL подключения к базе данных.
- Класс `Config` объединяет все настройки проекта, включая секретный ключ и флаг отладки.
- Функция `load_config()` читает переменные окружения и возвращает объект класса `Config`.

Логирование

Логирование помогает отслеживать работу вашего приложения. Оно особенно важно в продакшн-режиме для диагностики ошибок и анализа поведения системы.

1. Базовая настройка логирования

В файле `main.py` можно быстро настроить логирование с помощью стандартного модуля `logging`:

```
# main.py
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

@app.get("/")
def read_root():
    logger.info("Handling request to root endpoint")
    return {"message": "Hello, MIREA!"}
```

2. Продвинутая настройка логирования Чтобы логирование было более гибким, создадим отдельный файл `logger.py`:

```
# logger.py
import logging

def setup_logger():
    logger = logging.getLogger("my_app")
    logger.setLevel(logging.DEBUG)

    # Создаем консольный обработчик
    ch = logging.StreamHandler()
    ch.setLevel(logging.DEBUG)

    # Задаем формат логов
```

```

    formatter = logging.Formatter("%(asctime)s - %(name)s - %(levelname)s
    - %(message)s")
    ch.setFormatter(formatter)

    # Добавляем обработчик к логгеру
    logger.addHandler(ch)

    return logger

logger = setup_logger()

```

Теперь `logger` можно импортировать в любом модуле, например, в `main.py`:

```

# main.py (продолжение)
from app.logger import logger

@app.get("/db")
def get_db_info():
    logger.info(f"Connecting to database: {config.db.database_url}")
    return {"database_url": config.db.database_url}

```

Структура проекта

Вот пример новой структуры проекта, где конфигурация и логирование вынесены в отдельные файлы:

```

my_project/
├── app/
│   ├── __init__.py
│   ├── main.py           # Основной файл приложения
│   └── config.py         # Конфигурация проекта (загрузка .env,
dataclasses)
│   ├── logger.py        # Настройка логирования
│   └── models.py        # Модели данных (например, Pydantic-модели)
├── .env                 # Файл для переменных окружения
└── requirements.txt     # Список зависимостей

```

Конфигурация для продакшн-режима

При подготовке к продакшну важно учитывать дополнительные аспекты:

- **Отслеживание ошибок и уведомления:** Используйте такие сервисы, как Sentry, для мониторинга ошибок.
- **Параметры безопасности:** Настройте CORS, авторизацию и аутентификацию.

- **Использование HTTPS:** В продакшн-режиме обязательно применяйте SSL-сертификаты для шифрования данных.

Вы можете управлять этими параметрами через переменные окружения или создать отдельный конфигурационный файл для продакшна.

Пример настройки режима отладки в `main.py`:

```
# main.py (продолжение)
from app.config import load_config

config = load_config()

if config.debug:
    app.debug = True
else:
    app.debug = False
```

Линтеры и форматтеры

Чтобы код оставался чистым и соответствовал стандартам, используем автоматические инструменты форматирования и проверки:

- **Black** — автоформаттер кода.
- **Flake8** — линтер для проверки стиля кода.
- **isort** — сортировка импортов.
- **pre-commit** — инструмент для запуска этих проверок перед коммитом.

Обновленная структура проекта

```
my_project/
├── app/
│   ├── __init__.py
│   ├── main.py           # Основной файл приложения
│   └── config.py         # Конфигурация проекта (загрузка .env,
dataclasses)
│   ├── logger.py        # Настройка логирования
│   └── models.py        # Модели данных (например, Pydantic-
модели)
├── .black                # Конфигурация Black
├── .pre-commit-config.yaml # Конфигурация pre-commit
├── .flake8               # Конфигурация Flake8
├── .isort.cfg            # Конфигурация isort
├── pre-commit            # Скрипт для pre-commit хуков, который
можно запустить через команду `bash pre-commit`
├── .env                  # Файл для переменных окружения
└── requirements.txt      # Список зависимостей
```

Установка инструментов

Сначала установим все необходимые библиотеки:

```
pip install black flake8 isort pre-commit
```

Примечание: чтобы все работало корректно, версии библиотек такие:

```
black==24.8.0  
flake8==7.1.1  
isort==6.0.0b2  
pre-commit==3.8.0
```

Настройка Black

Создайте файл `.black` с настройками:

```
[tool.black]  
line-length = 120  
target-version = ['py311'] # тут пример под питон 3.11  
extend-exclude = '''  
    (venv,  
     alembic,  
     docker,  
     .git,  
     __pycache__)  
    '''
```

Запуск вручную:

```
black .
```

Black автоматически отформатирует код.

Настройка Flake8

Создайте файл `.flake8`:

```
[flake8]  
exclude = __init__.py,venv,alembic,docker,.git,__pycache__  
max-line-length = 120  
ignore = E203, E266, E501, W503
```

Запуск вручную:

```
black .
```

Этот инструмент укажет на стилистические ошибки в коде.

Настройка isort

Создайте файл `.isort.cfg`:

```
[tool.isort]
profile = "black"

[settings]
include_trailing_comma = true
line_length = 120
lines_after_imports = 2
skip = __init__.py,venv,alembic,docker,.git,__pycache__
```

Запуск вручную:

```
isort .
```

`isort` автоматически отсортирует импорты.

Настройка pre-commit

Создайте файл `.pre-commit-config.yaml`:

```
repos:
- repo: local
  hooks:
    - id: isort
      name: isort
      entry: isort --settings-file ./isort.cfg .
      args: ["--profile", "black"]
      language: system
      types: [python]
      pass_filenames: false

    - id: black
      name: black
      entry: black --config ./black .
      language: system
      types: [python]
      pass_filenames: false
```

```
- id: flake8
  name: flake8
  entry: flake8 --config .flake8 .
  language: system
  types: [python]
  pass_filenames: false
```

Создайте файл `pre-commit`:

```
isort --settings-file ./isort.cfg .
black --config ./black .
flake8 --config .flake8 .
```

Теперь можно запустить ручную баш скрип выше (`bash pre-commit`), чтобы линтеры и форматтеры прошли по коду проекта и внесли в него редакционные правки.

Подключение pre-commit хуков:

```
pre-commit install
```

Теперь перед каждым коммитом будут автоматически запускаться все проверки.

Made with ❤️ by **dvOretsky**